

Constraining Concepts Overload Sets

ISO/IEC JTC1 SC22 WG21 P0782R2

ADAM David Alan Martin (adam@recursive.engineer)
Erich Keane (erich.keane@intel.com)
Sean R. Spillane (sean@spillane.us)

Abstract

This proposal outlines a set of improvements to C++ Concepts that permits reasoning about Concepts in an interface-like manner. Under this proposal, function calls on variables constrained by a Concept will be prevented from resolving to some overloads that are irrelevant to the description of the Concept's interface. This proposal also adds a set of implicit Concept deduction rules for types which are returned from functions. These implicit rules create a system by which these altered overload resolutions apply in various equivalent pieces of code.

Introduction

The central purpose of Concepts is to simplify generic programming such that it is approachable to the non-expert developer. In general it makes great strides towards this end particularly in the capacity of invoking a generic function; however, the Concepts design does not deliver on this promise in the implementation of a generic function. This is because the feature does not constrain the overload set of a template-concept function itself. This is contrary to the expectations of non-experts, because to them Concepts should strongly resemble the callable properties of an interface. This mental model drives their expectations to believe that Concepts offer a mechanism to limit the set of operations which would be visible from within their constrained function to those which are specified by concept used by the constrained function.

The fact that this is not the case in constrained functions can lead to surprising violations of the author's expectations thereof. Unfortunately, this oversight cannot be corrected later. To correct this later would entail silent behavioral changes to existing code after the release of Concepts in a standard. In other words, this is our only chance to get this right.

First, we'll reiterate a list of solution variations that we're not proposing. We've debated this topic for years, and many potential solutions have been explored. We want to highlight that the solution that we're proposing is a novel idea, not one of these earlier ideas in disguise. Next, we'll outline improvements that the committee asked for and discuss how we might effect those results. Then, we'll explain our much-revised proposal and how it, we believe, solves the problem. Later we'll discuss the original proposal from P0782R1, leaving it mostly as it was, including motivations and examples, but correcting some errors and highlighting how this expanded proposal affects such examples. Finally, we'll anticipate common questions and answer them, before moving onto some design philosophy and future work and then conclusions.

What This Paper is NOT Proposing

Before we outline our amended proposal, we would like to quickly review a list of things that we are *not* proposing. All of these things have been highly contentious in the past, at best, and have led to undesirable consequences in some cases.

1. Any requirement on optimizers to make any aspect of the code generated by this solution more efficient
2. Definition Checking
3. C++0x Concept Checking
4. C++0x Concept Maps
5. The generation of "invisible" proxy types
6. The generation of "invisible" inline proxy functions
7. Dynamic dispatch
8. Function call tables
9. Implicit generation of adaptors
10. Any form of extra runtime code generation
11. Any form of extra compiletime type generation
12. Encoding concepts into the type system.

The mechanisms we're proposing are ways that the compiler can use existing information it *already has* to understand how to propagate constraint information or, when needed, how to rediscover missing constraint information from already available

information.

Overload Resolution is Insufficient

At Rapperswil, 2018, P0782R1 was discussed, in EWG. Our simple example regarding a potentially unanticipated overload to a function found by ADL was discussed in detail. As discussed in earlier papers and meetings, our problem is not with ADL per-se, but rather with the fact that "constrained functions" are only constrained in the sense that callers cannot inappropriately call them, but not that these functions are actually constrained in what they may call. Our proposal attempted to use modifications to the overload resolution rules to achieve these ends. Discussion in committee revealed this to be insufficient in a number of cases.

A simplification of our original example code (without the complete context) is:

```
namespace AlgorithmLibrary
{
    template< ConceptLibrary::Stringable S >
    void
    fire( const S &s )
    {
        std::cout << s.toString() << std::endl;
    }

    template< ConceptLibrary::Stringable S >
    void
    printAll( const std::vector< S > &container )
    {
        // Our original proposal would (with certain assumptions made about the meaning of
        // `const S &s: container`) not call any free-function `fire` on `s` which was found
        // by ADL in the namespace for `decltype( s )`, because it wasn't used in the concept
        // definition.
        for( const S &s: container ) fire( s );
    }
}
```

The problematic cases discussed in committee can be boiled down to three variations of the core of the `printAll` function:

Variation 1:

```
for( const S &s: container ) fire( std::identity( s ) );
```

Variation 2:

```
for( const S &element: container )
{
    const auto &s= element;
    fire( s );
}
```

Variation 3:

```
std::for_each( begin( container ), end( container ),
    []( const auto &s ){ fire( s ); } );
```

The problems in our simple overload resolution rules that were exposed by these three refactorings of what *should* be equivalent code require that a solution to this problem be much broader than just overload resolution. Each of these three variations appear to be the simplest case of the three shortcomings of just modifying overload resolution rules:

Shortcoming 1: The concept information does not propagate through composition of function calls, such as `f( g( x ) )`. The `std::identity` function, in the place of `g` clearly highlights this. It would likely be surprising to a programmer if `f( std::identity( x ) )` were completely different to `f( x )`. In fact, it would probably cause `std::identity` to be far less useful than it would be for many circumstances. This transitivity of Concept concerns is present for more than just `std::identity`, but it concisely illustrates the case. In our original proposal, the actual concept information about `decltype( x )` would be lost upon evaluation of the expression `std::identity( x )`. Although the `decltype` is the same, the concept information is lost at compiletime; somehow we would have to revive this constraint information..

Shortcoming 2: The auto problem. The auto keyword performs type deduction but it doesn't necessarily propagate the concept information that is available at compiletime into the resulting variable. This probably is simple to remedy, but a simple remedy to this does not account for the use of auto in the case of a polymorphic lambda. The problem is that the concept information associated with an expression (in this case just a constrained variable) does not necessarily create a likewise constrained variable in the result. This would create another surprising case where a temporary reference to a value which is constrained by a Concept has lost its constraint. A call to `f( localReference )` would not match the results of a call to `f( constrainedVariable )` when `auto &localReference= constrainedVariable;`. This would be equally surprising in the language, we feel. Somehow we have to keep this constraint information alive further.

Shortcoming 3: The deep-template-expansion problem. The use of a polymorphic lambda in a call to an STL algorithm will be unable to access the constraint information on its parameter, as the function template which calls it is not constrained by any of the concept information known to the caller. How do we propagate the constraint information down into the polymorphic lambda for it to use? The two typical alternatives, build it into the type system, or build a wrapper object for the constrained value are both equally painful and unsuitable to use in C++. Yet the act of refactoring a loop to use a polymorphic lambda and an STL algorithm shouldn't potentially cause a major alteration in the semantics of that code. Somehow we have to reintroduce this constraint information further down the call stack.

All three of these shortcomings are interrelated, and the problem requires an integrated solution; however, the shortcomings exposed by each of the variants are different aspects of the same problem -- preserving concept information in C++ for use in later phases of assigning semantics to expression evaluation.

The Problem of Preserving Concept Metadata at Compiletime

As discussed above, clearly we require a way to preserve the concept information known at compiletime for use at various parts of the compilation that are unrelated to the immediate call where overload resolution rules performed the right selection.

A common solution employed in many languages would be to introduce some kind of wrapper object which "boxes" up a constrained value into a distinct type and preserves this information in the type system by letting the box it rides in inform semantics. This also has the advantage of letting this box act as the discriminator to control which overloads of a function or operator actually get called. In some languages this box might be type-erased as to its pure contents, becoming essentially a C++ class with virtual functions. This is probably inappropriate for use in C++ as it sacrifices all knowledge of the underlying type at compiletime, thus preventing myriad optimizations.

Alternatively the boxing type could be a distinct wrapper for each underlying type, preserving the true type information throughout the compile process. The downsides of this are well known. This is what concept maps were in C++0x. We reiterate that we are *not* proposing these. We merely mention them here to explore the possibilities and to remind the reader that we've avoided this route.

A more subtle solution would use the type system a bit differently. Instead of building the concept information into the type being constrained when passing it down to lower function templates, we silently pass it down into the lower function's instantiation for use therein, and then we mangle the names of those function templates in order to encode the fact that that implementation may differ from any other differently constrained instantiation or an unconstrained instantiation. This seems tenable at first glance, but it tends to have some very surprising effects.

Consider the following function template: `void add_last_one( T, U );`. Although we wouldn't be making `T` or `U` into a box, both `T` and `U` were `std::list< ... >` and we passed a `U` which was a constrained `std::list< ... >`, wherein, perhaps, the `.back` operation is const-only, we start to get into some very problematic territory. The problem is that when the `u.back()` operation runs, it will be a different one than `T` gets. The fact that one is const may affect the choice of performing move semantics, which may pessimize a function at best, or even give different behavior at worst. Now imagine that this function `add_last_one` is called by a member function of a templated container. Now that container's entire operation varies based upon constraints to its member functions. Eventually, we wind up with multiple implementations of critical member functions that differ in the operations they do, which could seriously jeopardize the integrity of the memory layout. It's almost as-if two different type implementations were fighting over the *same* memory.

This mightn't be a problem if memory layout were identical under all variations, but note that `std::vector< bool >` is a totally different layout to `std::vector< struct_containing_a_single_bool >`. Further, consider that `add_last_one` might be a critical low-level library function such as `uninitialized_copy` or `std::copy` used in the implementation of `std::vector< ... >::reserve`. In order to prevent two different implementations of `::reserve` working on the same `std::vector< ... >`'s memory, it would become necessary to divorce their implementations all the way up to the outermost template. Thus we have sort of re-invented the concept map, just raised it up to a higher level -- push the concept map out to the broadest instantiated type using it. This also means that deep copy-in and deep copy-out semantics would become necessary as

the instantiated outer types can't be guaranteed to agree on layout and invariants, and forcing them to agree can lead to all sorts of problems.

Maybe it mightn't be so bad if we could limit the depth of constraint propagation to only a single, or a few levels. However, the same ugly problems of ODR violations and layout battles still await us here -- a function instantiated at "limit - 1" could have different behavior than "limit - 2", as the function at "limit" now would pass on constraint information one more level and thus change definitions again. Requiring the author to know how deep in a call hierarchy of templates a function is instantiated is a completely unviable option. And trying to make the compiler solve for some stable expansion which doesn't change meaning runs into the halting problem.

All of these problems arise from putting constraint information into the type system or plumbing it down through function implementations which eventually necessitate putting it into the type system. The problem is that putting things into the type system is kind of an all-or-nothing proposition -- either information is completely embedded therein, or it is completely absent.

Maybe the type system is the wrong place to put this kind of compiletime metadata. But where else might we put it?

Keep It Out of the Type System!

What if there were a way to make this kind of interface-like behavior work without touching the type system? What kinds of compromises might we have to make? If it's not in the type system, then where do we put it?

Obviously, the benefit of keeping it out of the type system is to avoid the problems that we outlined in our section "The Problem of Preserving Concept Metadata at Compiletime". However, the problem goes deeper than just keeping it out of the type system -- we can't preserve any of it beyond immediate use at a call site. If we preserved constraint information, carrying it into deeper template instantiations we'd merely reintroduce the divergent implementations problem (ODR issue) and then either try to solve it with name mangling (basically the type system) or by just hoping that it's not a big problem (which it certainly will be a big problem for someone!)

Of course losing the concept information is what we were trying to prevent, so how do we preserve type information while not preserving it? We don't! We instead rely upon *reintroducing it* at various points in expression checking. In other words, we accept and embrace the fact that constraint information evaporates across any function call, but we find ways of reacquiring it from contextual clues, hints to the compiler, definitions of concepts, and declarations of functions. Our solution relies upon introducing a few new keywords to the language, with some mostly heretofore-unseen semantics and then defining a few implicit injections of those new grammatical productions. This tactic to reintroduce the constraint information preserves single compilation of lower-level functions, and it prevents the explosion of types and mangling that is necessary when recompiling lower-level functions.

The end result, we believe, is a reasonable compromise position. The resulting syntax for users wishing to preserve constraint information will be nearly the same, if not the same, in most cases.

Before explaining the syntax and introducing the new keywords, we'd like to show the 3 variations of the problem again and contrast them with the modified form in our expanded proposal:

Variation 1:

```
// The problematic variation:
for( const S &s: container ) fire( std::identity( s ) );

// Which under our expanded proposal needn't change at all!
for( const S &s: container ) fire( std::identity( s ) );
```

It is pretty evident that our revised solution delivers the expected clarity of expression for transitivity of concept information across function calls, if this problematic case isn't even a problem anymore.

Variation 2:

```
// The problematic variation:
for( const S &element: container )
{
    const auto &s= element;
    fire( s );
}
```

```
// Which under our expanded proposal might not need to change, depending upon some direction
// from the committee, but it would become, in the worst case:
for( const S &element: container )
{
    const __new_keyword_like_auto__ &s= element;
    fire( s );
}
```

The addition of a new keyword, similar to auto for use in constraint propagation certainly bodes well for the expressivity, and the possibility to even use auto itself for this case is appealing. Clearly our solution makes most forms of constraint reintroduction nearly painless to use.

Variation 3:

```
// The problematic variation:
std::for_each( begin( container ), end( container ),
    []( const auto &s ){ fire( s ); } );

// This kind of rewrite would be possible under our solution (which was also a possibility in the
// earlier overload-resolution-only solution):
std::for_each( begin( container ), end( container ),
    []( const Concept &s ){ fire( s ); } );

// Or, another possible rewrite (assuming `container` is something like `std::vector< Concept >`):
std::for_each( begin( container ), end( container ),
    []( const __new_keyword_like_decltype__( container.front() ) &s ){ fire( s ); } );
```

The solution to the third shortcoming is the most disappointing, as preserving the concept through `std::for_each` for consumption by auto was strongly desired by both the committee and by the authors. We will justify in our conclusions why we're not as disappointed as we might have been, however. Additionally, we have some potential future directions of exploration that may make the original syntax possible again in some circumstances.

How to Reinvent Lost Concept Constraints

In order to make it easier to understand this new battery of features, we will introduce them in a specific order, and build each from earlier primitives, using "as-if" and "almost as-if" explanations of their meanings. Most new "keywords" we will introduce are actually sequences of existing keywords chosen to convey a meaning and also to be ugly. We don't particularly care the final result of the naming debate on these new keywords, just that facilities exist with the semantics we will describe below, under some names.

In the process of this exploration, we will have to make our examples look worse, before they get better. The introduction of all of this new machinery permits rewriting the problematic expressions in terms of this new machinery, so as to replace the constraint information. Then, a later section will introduce a number of implicit ways to deduce the need for and the correct generation of this machinery. After exploring the implicit deduction of the machinery, we shall see how the expanded examples can have their explicit use of this new syntax removed and wind up with code that is usually the same as the code in a problematic example which actually works. (Essentially we are defining tools we can use to explain how to make those examples work.)

First, we will start with a new kind of cast, `concept_cast`. Unlike the rest of the new "keywords" we will introduce we feel that this particular one is uncontroversial and shouldn't spark much naming debate, thus we've proposed a reasonable name. It should be, hopefully, fairly self-evident what it does, but this is how we define it:

```
concept_cast< const Concept & >( someVariable )
```

Would be almost as-if the following code were written, instead:

```
[&]() -> const Concept & { return []( const Concept &c ){ return c }( someVariable ); }()
```

In other words, `concept_cast` is a cast which statically checks for concept candidacy and asserts this of the result. It is not used to lie about something being a model of a concept, unlike other casts such as `reinterpret_cast`. The cast needn't be implemented by rewriting the expression to the specified lambda form, but it should be semantically similar, in that the result of such a cast is considered to be a constrained value.

Now with this primitive, we can start to explore some potential solutions to some of the problems we have, especially the transitivity (shortcoming 1) problem:

Shortcoming 1 Code

```
// The problematic variation:
for( const S &s: container ) fire( std::identity( s ) );

// How we might use `concept_cast` to fix it:
for( const S &s: container )
{
    fire( concept_cast< const S &>( std::identity( s ) ) );
}
```

Well, clearly `concept_cast` hasn't made the situation any better in terms of syntax, but it does provide a mechanism to force the compiler to reanalyze whether a value matches a concept and to reintroduce a constraint on an expression. We will need to go further, but it is worth pointing out that this `concept_cast` has some semantic overlap with a few of the proposals that ask for concept-deduced `auto` and similar things.

Using `concept_cast` we can go further and introduce a new keyword `decltype for concept`. Hopefully its meaning is mostly self-evident. This keyword is used thus:

```
concept_cast< decltype for concept( someConstrainedVariable ) &>( unconstrainedExpression );
```

Semantically, it is a type deduction keyword which gets the concept name for a constrained variable and lets it be used in contexts such as:

```
decltype for concept( someConstrainedVariable ) &reference= someConstrainedVariable;
```

This lets us start preserving concept information, albeit manually, in local variables. This is also somewhat related to concept-deducing proposals that already exist. We welcome these proposals and have no preference on their syntax, just that the above to capabilities, `decltype for concept` and `concept_cast` are also provided under some name or another. With this new keyword, `decltype for concept` we can rewrite our second problematic example to not have a problem, but be rather ugly:

Shortcoming 2 Code:

```
// The problematic variation:
for( const S &element: container )
{
    const auto &s= element;
    fire( s );
}

// how we might use `decltype for concept` to fix it:
for( const S &element: container )
{
    const decltype for concept( element ) &s= element;
    fire( s );
}
```

It should be noted that `decltype for concept( element )` is ill-formed if `element` has no associated constraint. It also should be noted that the variable `s` has the associated constraint which was deduced. It is in this manner that we will see the reintroduction of associated constraints -- through various casting and concept deduction mechanisms we introduce. Eventually we'll show how many of them can become implicit as well!

Clearly with these two primitives, an equivalent for `auto` in the concept space becomes somewhat desirable and possible. This is somewhat distinct from most constrained `auto` proposals, as we're asking the compiler to propagate an unmentioned constraint. This new keyword we call `auto for concept`, and its usage looks like this:

```
const auto for concept &reference= constrainedVariable;
```

Given our existing keywords, we can explain this keyword using as-if semantics. The above usage is the same as if we'd written:

```
const decltype for concept( constrainedVariable ) &reference= constrainedVariable;
```

Clearly this permits a better rewrite of the second problem:

Shortcoming 2 Code:

```
// The problematic variation:
for( const S &element: container )
{
    const auto &s= element;
    fire( s );
}

// how we might use `decltype for concept` to fix it:
for( const S &element: container )
{
    const auto for concept &s= element;
    fire( s );
}
```

If the `auto for concept` keyword winds up being somewhat easy to type, the above rewrite is pretty close to the original intent. We also suggest that `auto for concept` be implicitly `auto for concept` in circumstances which have associated concept information; however, as `auto for concept` is ill-formed in uses on non-constrained right-hand expressions, we point out that the use of `auto for concept` should probably be favoured in circumstances where concept preservation is important. With that noted, some may feel that overloading `auto` to be a relaxed form of `auto for concept` without the ill-formed-if-not-constrained requirement is potentially misleading. We are content to leave this point open to committee discussion. In either case, we believe that our proposal is self consistent and usable. This discussion point is merely about whether the second problematic variation itself need be problematic. Some may find this extension of `auto` inappropriate.

In all of this introduction of keywords, we've not significantly addressed the third kind of problem. With our current enhancements, let's explore it:

Shortcoming 3 Code:

```
// The problematic variation:
std::for_each( begin( container ), end( container ),
    []( const auto &s ){ fire( s ); } );

// This kind of rewrite would be possible under our solution (which was also a requirement of the
// earlier solution):
std::for_each( begin( container ), end( container ),
    []( const Concept &s ){ fire( s ); } );

// One other possible rewrite can now use `decltype for concept`, which was our mystery keyword
// before:
std::for_each( begin( container ), end( container ),
    []( const decltype for concept( container.front() ) &s ){ fire( s ); } );

// However, this rewrite actually is ill-formed:
std::for_each( begin( container ), end( container ),
    []( const auto for concept &s ){ fire( s ); } );
```

In the ill-formed case the `auto for concept` keyword has no concept information to use in the `std::for_each` function, except potentially any constraints on the τ type over which the iterators travel... but such a type should be unconstrained. If `std::for_each` were something like `std::transform`, then `auto for concept` would deduce `Copyable` for instance, and would probably cause compile failures when functions that are not part of the `Copyable` interface are called. Basically `auto for concept` on a lambda will either deduce a potentially more constrained concept than the one used by the function in which the lambda is defined, or it will cause a compile failure as `auto for concept` is ill formed when there is no concept data to propagate in from the immediate calling site. We also suggest that a Quality of Implementation concern could be to provide compiler warnings for unconstrained polymorphic lambdas which are defined in constrained functions. With all of these points covered, this is about as far as we can reclaim the territory of reinventing concept information for lambdas which are passed to lower functions. Either one explicitly specifies the constraint (possibly using `decltype for concept`), or one asks for `auto for concept` and is content with a more constrained concept or a compile failure. Pushing the concept information down into the template for the general case becomes a major nightmare, as discussed in earlier sections.

Returning to the composition of functions case, such as `std::identity`, we see that thus far, we haven't delivered on the promise to make that code clean (let alone make it so easy to use that it's invisible).

Shortcoming 1 Code

```
// The problematic variation:
for( const S &s: container ) fire( std::identity( s ) );
```

```

// How we might use `concept_cast` to fix it:
for( const S &s: container )
{
    fire( concept_cast< const S &>( std::identity( s ) ) );
}

// Now we might get a bit cute and do this, but it's different code:
for( const S &s: container )
{
    const auto for concept &tmp= concept_cast< decltype for concept( s ) >( std::identity( s ) );

    fire( tmp );
}

```

This brings us to the last new keyword and mechanism, concept transformers, or constraint replacers. The (incredibly ugly) keyword we introduce for this is using `this typename< ... >` for concept return. We'll outline a use of it:

```

template< typename T >
const T &
identity( const T &t )
    using this typename< decltype for concept( t ) > for concept return
{
    return t;
}

```

This new keyword does NOT affect the meaning of this function itself. It is not part of the function's type. It is not part of the function's definition. Leaving it off in some TUs will not cause an ODR violation within the definition of `identity`. It is, instead, a kind of compiler hint for callers of this function. Despite the fact that disagreement over the presence of this this new constraint replacer syntax isn't an ODR violation, it shouldn't be left off of functions like the `identity` function as the constraint replacer syntax affects how callers will call this function... and that will lead to ODR violations. This distinction between affecting the definition and affecting the caller will become extremely important.

The fact that this decoration does not affect the definition of `identity`, but instead affects the callers, might seem a bit baffling at first. What exactly does it mean then? Let's assume that the above `identity` is `std::identity`. In that case, the following expression:

```
std::identity( constrainedVariable )
```

can be interpreted almost as-if it were rewritten to be:

```
concept_cast< decltype for concept( constrainedVariable ) &>( std::identity( constrainedVariable ) )
```

The only major difference is that in expressions which return r-values, the implicit `concept_cast` does not interfere with lifetime extension, whereas an explicit invocation would.

In other words, an implicit `concept_cast` is invoked on the results of the `identity` function and the concept given in the `< ... >` of the using `this typename` for concept return constraint replacer is plugged into that `concept_cast`. With this primitive, we can imagine decorating the ENTIRE STL and all other codebases with constraint replacers. Since the constraint replacer syntax is also not directly tied to a template declaration, it may be placed on non-template functions as well, with the same effect. For example:

```
int identity( int x ) using this typename< decltype for concept( x ) > for concept return { return x; }
```

Now concept-based constrained callers to this non-templated `identity` will find that constraints are preserved across this call. This generalized solution can be adapted for multiple argument functions, and the `< ... >` syntax should permit the specification of arbitrary compiletime metaprograms to determine what concept is returned. In our estimation, however, most such metaprograms will be "concept identity" of some parameter or another, or possibly "concept union" of all parameters.

This last mechanism permits, when proper constraint replacers are written for functions like `std::identity`, a preferred interpretation of our first problematic case:

Shortcoming 1 Code

```

// The problematic variation now is actually just fine!
for( const S &s: container ) fire( std::identity( s ) );

```



```
// What the above actually expands to, in an almost as-if, fashion.
for( const S &s: container )
{
    fire( concept_cast< decltype for concept( s ) >( std::identity( s ) ) );
}
```

Where the `concept_cast` was implicitly injected because of the constraint replacer which was defined as part of `std::identity`'s signature.

If we imagine such a world, where every function has been given an appropriate constraint replacer then all transitive function calls, such as `f( std::identity( x ) )` will have the constraint replaced after it was lost. However, boiling the oceans to arrive at such a world would be a big problem... If only there were a way to get there sooner?

Implied Constraint Replacement

The problem of asking everyone to stop the world and add constraint replacers seems insurmountable, yet there is actually a major path forward here -- we can infer the need for and definitions of constraint replacers in many circumstances.

From Concept Definitions

The simplest case to infer a constraint replacer is by looking at the definition of a concept. Consider:

```
template< typename T >
concept Serializable= requires( T instance )
{
    { instance.toString() } -> StringLike;
};
```

From that concept's definition we can infer that all calls to `.toString()` on anything that is a `Serializable` should have an implicit `concept_cast< StringLike & >( instance.toString() )` wrapping them, or more specifically, we can imply the existence of using this `typename< StringLike > for concept return` as a constraint replacer on the `toString()` function.

From Class Template Members

Another simple case where we can infer a constraint is on members of templated classes. Consider:

```
template< typename T, std::size_t sz >
class array
{
private:
    T stuff[ sz ];

public:
    T &operator[] ( const std::size_t idx ) { return stuff[ idx ]; }
};
```

Without rewriting this code at all, it is almost certainly safe to imply that `array< MyConcept, 42 >::operator[]` has a constraint replacer defined as using this `typename< decltype for concept( T ) > for concept return`. This permits most template libraries to automatically work as-if they were designed for this all along. The reason that this is safe is that all callers to this function which have concept constraints on `array< ... >` already know that constraint and were compiled with it, binding that constraint replacement to the function which knows about the constraint. As functions which don't know about constraints yet are all that there is, we won't break any existing code, as the definition of `operator[]` is unchanged.

With the appropriate amount of propagation of constraints that are parameters to constraints, it may be possible to properly constrain operations on related types, such as iterators. For example:

```
template< typename Element >
class vector
{
public:
    iterator begin() const;
    T &operator[] ( const std::size_t idx );
};
```

If `RandomAccessContainer< T >` requires that `begin()` return `RandomAccessIterator< T >`, and `RandomAccessIterator< T >` requires that operator `*` return `T &`, then the results of iteration can be implicitly deduced to have the same constraint as the constraint on the container elements themselves. This transitivity of constraint application permits containers and similar objects to function correctly in constraint replacement situations. Most of this information is known to the compiler already and it often does perform the concept tests for these situations, during constrained template instantiation.

From Function Templates Which Explicitly Return Concepts

When new functions will be written that use "natural" syntax for concepts, we will have more constraint information available to work with. Consider a modern max function:

```
LessThanComparableValue
max( LessThanComparableValue a, LessThanComparableValue b )
{
    return a < b ? a : b;
}
```

We should obviously imply the existence of at least using `this typename< LessThanComparableValue > for concept` return as a constraint replacer.

From Function Template Arguments

Many function templates are probably discriminators or combinators over the same concept. This means that we could probably infer them from the function signatures. Consider a legacy max function:

```
template< typename T >
T
max( T a, T b )
{
    return a < b ? a : b;
}
```

Without analyzing the function body itself, it is probably safe to assume that using `this typename< decltype for concept( a ) | decltype for concept( b ) > for concept` return type is a constraint replacer for this function. We know that both types are the same, and that they likely share the same constraint when passed. Even if they do not, the disjunction mechanism in concepts permits accomodating this. It is worth discussing if the implicit constraint replacer should be suppressed if `decltype for concept` of `a` and `b` disagree. In the single argument case, like `identity`, it probably naturally applies. There is some room for discussion as far as what the correct set of rules for implicit constraint replacers on function templates should be; however, we strongly encourage some set of rules as it will greatly ease the burden on existing library maintainers. We see this situation as analagous to implicit deduction guides. We need to pick the right answer, and then the libraries mostly will benefit from the implicit guides' correct behavior.

From Non-Template Function Arguments

Some template functions have non-template overloads that invisibly participate in the same overload set. Consider:

```
int identity( int x ) { return x; }
```

Asking those functions to explicitly add using `this typename< decltype for concept( x ) > for concept` return just to play nicely is going to get annoying. We should decide whether we want to make this implicit requirement apply to just single argument non-template functions or to multiple arguments. Care needs to be taken here in selecting the right level of implicit generation, and some discussion is probably necessary.

There is also the consideration of whether some implicit forms should dominate over non-implicit forms, if ever. We suggest that the constraint replacers that are implicitly generated from concept definitions for expressions should always dominate over all other constraint replacers. We also suggest that explicit constraint replacers should dominate over any implicitly generated constraint replacers for all other cases.

We also suggest that a keyword such as using `this typename< delete > for concept` return exist to explicitly disable constraint replacers from being implicitly generated for a function.

With these rules providing a sensible set of defaults we should be able to make most existing code play well with this overload resolution scheme and constraint propagation. We suggest that while all of these implied constraint replacers could be useful,

the very useful ones are those deduced from templates, and the most useful will be those deduced from concept definitions.

Tying the Pieces Together

Now with all of this new machinery, we can show how each of the three problematic variations becomes either much easier or how the problem evaporates entirely.

Variation 1:

```
for( const S &s: container ) fire( std::identity( s ) );  
// The above code should just work, assuming either manual insertion of constraint replacers,  
// or implicit generation of constraint replacers for function templates like `std::identity`
```

Variation 2:

```
for( const S &element: container )  
{  
    const auto for concept &s= element;  
    fire( s );  
}  
// By using an explicit `auto` created for concepts we get the right behavior. We also can broaden  
// the rules on `auto` to permit it to be equivalent to `auto for concept`, for constraint associated  
// cases.
```

Variation 3:

```
std::for_each( begin( container ), end( container ),  
    []( const Concept &s ){ fire( s ); } );  
  
std::for_each( begin( container ), end( container ),  
    []( const decltype for concept( container.first() ) &s ){ fire( s ); } );  
  
// This is the only case that really remains hard. One has to either explicitly name the concept  
// being passed, or has to deduce it from a local variable which already has the constraint associated  
// with it. Both cases require bundling the constraint with the lambda before it is shipped off to  
// the algorithm for use.
```

In this world, assuming a compiler which warns on `[]( const auto & )` lambdas created in constrained functions, we can mostly program safely, in a fashion similar to that we already are used to doing. However, a few new keywords were needed, or existing ones had to learn a few new tricks. The most disappointing case is that polymorphic lambdas being passed into algorithms cannot get the concept constraints shipped onto them; yet, we don't see this as too much of a burden. Although this limitation directly conflicts with the almost-always-auto paradigm (which is not a universally accepted paradigm), we suggest that this paradigm is merely a step along the way to even better coding styles. In the past, we always dealt with a concrete type system, as it was a safer alternative to completely untyped languages. Then we adapted static duck-typing into the language such that we wanted to write more generic code without being concerned about specific types, while remaining typesafe. Then we found that there were benefits to the constraints provided by types and we sought to constrain our generic code with concepts. Part of that next phase of evolution is getting rid of broad type deduction in favour of constrained type deduction or writing in terms of constrained types. This trend, if followed, will move people away from `[]( const auto & )` style lambdas and towards `[]( const Concept & )` style lambdas... which is where our proposal sort of requires us to go already.

The `auto` in polymorphic parameters shouldn't be sometimes constrained and sometimes not -- such a sneakily dual-meaning keyword will cause endless confusion. For this reason, it may be worth it to not make `auto` ever be aware of constraint information, as surprising results will ensue when code only sometimes respects the constraints. However, high quality compilers may be able to issue warnings for bad (unconstrained) uses of `auto` in constrained functions, thus closing the loop as a QoI concern.

Our New Proposed Overload Resolution Rules

In our original paper, our overload resolution rules helped to resolve ambiguities among multiple overloads in a function. Unfortunately the way that we had to do that was to always accept the best-match for a function as a supplement for the overload resolution. This makes move semantics and other type system related optimizations become impossible. We have changed the overload resolution rule consider the entire viable set, not just the best match, in concept match when determining

what overloads in a type which models a concept are suitable overload candidates for a specific call site. This has the advantage of mostly solving the "implicit expression variants" problem, such as r-value vs. l-value overloads.

We propose a moderate alteration of the overload resolution rules and name lookup rules that statically filters out some overloads based upon whether those functions are used to satisfy the concept's requirements. This constrained overload set hides some functions from visibility in the definition of constrained functions. No change should be necessary to the rules of name lookup itself; however, our new overload resolution rules will affect the results overload resolution on unqualified name lookup in constrained functions-- this is by design. We also suggest that it might be desirable to borrow a keyword (we suggest `explicit`) to indicate that this amended lookup rule should be followed. The need for this keyword to enable these lookup rules will be discussed further in the "Design Considerations" section of this paper.

Specifically, our design is to change overload resolution to be the following (taken from cppreference.com's description of the overload resolution rules):

Current overload resolution	Desired overload resolution
Given the set of candidate functions, constructed as described above, the next step of overload resolution is examining arguments and parameters to reduce the set to the set of viable functions To be included in the set of viable functions, the candidate function must satisfy the following: 1. If there are M arguments, the candidate function that has exactly M parameters is viable 2. If the candidate function has less than M parameters, but has an ellipsis parameter, it is viable. 3. If the candidate function has more than M parameters and the M+1'st parameter and all parameters that follow must have default arguments, it is viable. For the rest of overload resolution, the parameter list is truncated at M. 4. If the function has an associated constraint, it must be satisfied (since C++20) 5. For every argument there must be at least one implicit conversion sequence that converts it to the corresponding parameter. 6. If any parameter has reference type, reference binding is accounted for at this step: if an rvalue argument corresponds to non-const lvalue reference parameter or an lvalue argument corresponds to rvalue reference parameter, the function is not viable.	Given the set of candidate functions, constructed as described above, the next step of overload resolution is examining arguments and parameters to reduce the set to the set of viable functions To be included in the set of viable functions, the candidate function must satisfy the following: 1. If there are M arguments, the candidate function that has exactly M parameters is viable 2. If the candidate function has less than M parameters, but has an ellipsis parameter, it is viable. 3. If the candidate function has more than M parameters and the M+1'st parameter and all parameters that follow must have default arguments, it is viable. For the rest of overload resolution, the parameter list is truncated at M. 4. If the function has an associated constraint, it must be satisfied (since C++20) 5. If at least one of the arguments to the function is constrained and that function was found by unqualified name lookup (without <code>::</code> in the name of the function) and the lookup found a name that is otherwise not visible at the calling location, then that function is only viable if that function was part of the viable set of overloads which were necessary to satisfy the argument's concept constraint. (this paper, amended from P0782R1) 6. If at least one of the arguments to the function is constrained and that function was found by qualified name lookup (with <code>::</code> in the name of the function) and the lookup found a name that is otherwise not visible at the calling location, and the name is mentioned in qualified form (using <code>::</code> in the name of the function) in all of the arguments' concept constraints, then that function is only viable if that function was part of the viable set of overloads which were necessary to satisfy the argument's concept constraint. (this paper, new to P0782R2) 7. For every argument there must be at least one implicit conversion sequence that converts it to the corresponding parameter.

8. If any parameter has reference type, reference binding is accounted for at this step: if an rvalue argument corresponds to non-const lvalue reference parameter or an lvalue argument corresponds to rvalue reference parameter, the function is not viable.

Our amended rule permits all members of the viable set of overloads from a concept match to continue to participate in overload resolution in actual usage. This permits l-value and r-value overloads of an argument to participate in overload resolution if the concept specifies a prvalue parameter in that slot. This also permits better match type overloads, such as `const char *`, `std::string`, and `std::string_view`, when all three could be used.

Our new rule addresses concerns from the committee that regarded the use of qualified names in concepts. The reason for the second rule is to only enforce this if qualified names are used in concepts for parameters. The authors feel that an explicitly qualified name, e.g. `std::min` when called on a constrained parameter should only obey constrained lookup if `std::min` is clearly part of the "interface" defined by that concept. Otherwise it's pretty evident that `std::min` is an explicit call to a specific function in a specific namespace.

Original Simple Motivating Example

In our original paper, this motivating example was to highlight some potentially unexpected problems with ambiguity. After discussion in Rapperswil, the authors are content to accept that the `LoanInterestSerializer` in the following example should probably be considered a poorly written class. The major change to the overload resolution rule that this relaxation makes is to consider the entire viable set, not just the best match, in determining what overloads in a type which models a concept are suitable overload candidates for a specific call site. This has the advantage of mostly solving the "implicit expression variants" problem, such as r-value vs. l-value overloads.

```
// Assume a concept, `IntegerSerializer` which requires a member function called `serialize` which
// takes an `int` and returns a `std::string` with the representation of that integer, in the format
// appropriate to the implementation.
```

```
template< typename Instance >
concept IntegerSerializer= requires( Instance instance, int value )
{
    { std::as_const( instance ).serialize( value ) } -> std::string;
}
```

```
class SimpleSerializer
{
    std::string serialize( int value ) const;
};
```

```
struct PrecisionSerializer
{
    std::string serialize( double preciseValue ) const;
};
```

```
struct LoanInterestSerializer
{
    std::string serialize( int interestBasisPoints ) const;
    std::string serialize( double interestRate ) const;
};
```

```
// This is a terse-syntax style template function. Whether the author knew he or
// she was writing a template is not necessarily clear.
std::string formatLogarithmicValue( IntegerSerializer &serializer, int integerValue )
{
    return serializer.serialize( std::log( integerValue ) );
}
```

The issue is in what kind of object we pass to `formatLogarithmicValue`. If we pass an `SimpleSerializer`, then no surprises happen -- the double is implicitly converted to int. If we pass `PrecisionSerializer`, then the definition of the concept will pass and an implicit conversion to int will happen, which is potentially surprising; however, many programmers are reasonably comfortable with the idea of fundamental type conversions. The most surprising case is that of

LoanInterestSerializer. LoanInterestSerializer provides a double and an int overload. Although the concept requested function with a signature that accepts an int, the overload which accepts double in its signature will be called.

From the perspective of the compiler, the way it will actually compile the function formatLogarithmicValue is as if it were written:

```
template< typename IntegerSerializer >
std::string
formatLogarithmicValue( IntegerSerializer &serializer, int integerValue )
{
    return serializer.serialize( std::log( integerValue ) );
}
```

At this point, the invocation of IntegerSerializer::serializer will be whatever best matches decltype(std::log(integerValue)), which is the overload with double as its parameter. This is likely surprising behavior to the author of formatLogarithmicValue, as well as the caller of formatLogarithmicValue. Both of these authors would expect that the constraints described by the concept would be obeyed, yet paradoxically the overload which was not the best match for the constraint was actually the overload that was actually invoked in the body of the "constrained" function!

The only way for an author of such a constrained function to avoid this, at present, is to rewrite formatLogarithmicValue in such a way as to prevent the incorrect lookup. Unfortunately, this requires a level of C++ expertise regarding name lookup and overload resolution which is at odds with the level of expertise expected of the audience of Concepts, viz. the non-expert programmer. Such a rewrite might appear thus:

```
template< typename IS >
std::string
formatLogarithmicValue( IS &serializer, int integerValue )
    requires( IntegerSerializer< IS > )
{
    return std::as_const( serializer ).serialize(
        static_cast< int >( std::log( static_cast< double >( integerValue ) ) ) );
}
```

This does not appear to be code that would be expected of the audience targetted by Concepts. Additionally, although one of the authors of this paper is author of std::as_const, this is not the purpose nor audience he had in mind when he proposed it.

The primary purpose of the static casts and as-consts in this rewrite are actually dedicated to the selection of the correct overload, not to any specific need to have a value in the form of any specific type.

Although the above example is mostly unaltered from its original form, we have since been convinced that using concepts to perform overload discrimination which may avoid ambiguities is probably a bad idea. Concepts are specified in terms of expression, and thus expression compatibility should be the main concern, not types of parameters. It is very difficult to distinguish an undesired conversion from a desired conversion for performance reasons. In that vein, we have decided to err on the side of all conversions being desired. This means that the above example will not be remedied by this proposal. However, any remedy of that situation would risk eliminating user-defined conversions and overloads which may exist for efficiency purposes. To that end, we embrace the current wisdom that overloads of a function should "do the same thing" regardless of their parameter types.

An Example at Scale

In our original proposal we presented the following example as a motivating example of how independent authors will find surprising results from concepts, especially using terse syntax. Our proposal does indeed fix this example, and now we have addressed the concerns that were raised in Rapperswil over its limitations. We present this original motivating example, in its entirety, so that the reader who has understood our proposal may see how it will behave as the non-expert author intended.

In this example, distinct namespaces are used to highlight the fact that different pieces of code were written by different authors, in different code bases, perhaps at different times. The UserProgram namespace is representative of a final application author trying to tie together disparate libraries in C++ and produce a coherent, correct result from a program.

```
namespace ConceptLibrary
{
    template< typename Instance >
    concept Stringable= requires( Instance instance )
    {
        { std::as_const( instance ).toString() } -> std::string;
    }
}
```

```

    }
}

namespace PayrollLibrary
{
    class Employee
    {
        private:
            std::string name;

        public:
            Employee( std::string initialName );

            // This satisfies the Stringable concept, and the author of this type knows that.
            std::string toString() const;

            friend bool operator == ( const Employee &lhs, const Employee &rhs );

            // The following functions are friend, to indicate that ADL is intended.

            // Terminate the specified employee's employment
            friend void fire( const Employee &emp );

            // Initiate the specified employee's employment
            friend void hire( const Employee &emp );

            // Returns true if the specified employee is employed and false otherwise.
            friend bool worksHere( const Employee &emp );
    };
}

namespace AlgorithmLibrary
{
    // This "fires" off a stringable object to be processed. It is a terse-syntax style
    // template function. Whether the author knew he or she was writing a template is
    // not necessarily clear. Although it is likely that the author knew the function
    // was a generic function.
    void
    fire( const ConceptLibrary::Stringable &s )
    {
        std::cout << "I am interested in " << s.toString() << std::endl;
    }

    // This too is a terse-syntax style template function. Whether the author knew he or
    // she was writing a template is not necessarily clear. Although it is likely that the
    // author knew the function was a generic function. The purpose of this function is
    // to call the above helper function in a loop.
    void
    printAll( const std::vector< ConceptLibrary::Stringable > &v )
    {
        for( auto &&s: v ) fire( s );
    }
}

namespace UserProgram
{
    void code()
    {
        std::vector< PayrollLibrary::Employee > team;
        team.emplace_back( "John Doe" );
        AlgorithmLibrary::printAll( team );
    }
}

```

In this example the intent of the three separate authors is apparent. The author of the `PayrollLibrary` simply wished to afford his users the ability to represent employees at a company. The author of the `AlgorithmLibrary` wished to afford his users the ability to print printable things and needed to write an internal helper method to better organize his code. The author of `UserProgram` naively wished to combine these reusable components for a simple task. However, a subtle behavior of name lookup in function templates resulted in the termination of employees, rather than the intended call to an implementation detail. This happened because the name of the implementation detail happened to collide with the name of some irrelevant API in the `Employee` object being processed. Recall that the author of `AlgorithmLibrary` is likely unaware of the fact that types may exist which are `Stringable` and yet interfere with the name he chose for his internal implementation detail.

The authors of this paper recognize the importance of respecting the original intent of the programmers of these components without burying them in the details of defensive template writing. These kinds of examples will come up frequently and perniciously in codebases which import third party libraries and work in multiple groups each with different naming conventions. Even without variance in naming conventions, names that have multiple meanings to multiple people are likely to be used across disparate parts of a codebase, and thus they are more likely to exhibit this pathological behavior.

Addendum: The original example used `auto` as a loop variable deduction/declaration in the range-based `for` in `AlgorithmLibrary::printAll`. Our proposal may require that user to switch to `auto for` concept in that context, if `auto` is not enhanced to be `auto for` concept in constrained contexts; however, we believe that a high quality implementation can provide diagnostic compiler warnings for such code should that be the committee's decision.

Why it is Important to Address this Now

Should the terse syntax be accepted into the current standard, without addressing this issue, then future attempts to repair this oversight in the language specification, leave us with one of two incredibly unpalatable alternatives and one unsatisfying one:

1. Make constraint violations an error, thus requiring extreme verbosity.
2. Silently change the meaning of existing code, with ODR implications, because these constrained functions are templates.
3. Introduce a new syntax for indicating that a function definition should be processed in a manner which is more consistent with average programmer expectations;

In the first case, vast amounts of code will fail to compile in noisy ways. After that point, the user code would need to be rewritten, in a manner similar to the necessary rewrites as described above.

In the second case, massive fallout from ODR, silent subtle semantic changes, and other unforeseen dangers lie in wait.

In the third case, the benefits of the "natural" syntax are lost, as the best syntax for beginners is no longer the natural syntax! This obviously defeats the intended purpose of Concepts with a natural syntax.

Design Philosophy

There are other cases where current Concepts can fail to prevent an incorrect selection of operation. This fails to deliver upon a big part of the expected benefits of this language feature. The comparison has been drawn between C++ Virtual Functions and Concepts. As Concepts are being presented to bring generic programming to the masses, it is vital that 3 core safety requirements be considered. These requirements are similar to aspects of Object Oriented Programming.

1. An object passed to a function must meet the qualifications that a concept describes. This is analogous to how a function taking a pointer or reference to a class has the parameter checked for substitutability. The current Concepts proposal provides this extremely well.
2. An object written to be used as a model of a concept should have its definition checked for completeness by the compiler. This is analogous to how a class is checked for abstractness vs concreteness. The current Concepts proposal lacks this. However, this is approximated very well by the concept checking machinery. This guarantees that every class which is matched to concept provides a definition for every required operation under that concept, thus satisfying the requirements of the concept.
3. A constrained function is only capable of calling the functions on its parameters that are described by its constraining Concepts. This is analogous to how a function taking a pointer to base is only allowed to call members of the base -- new APIs added in any derived class are not considered to be better matches, ever. The current Concepts proposal lacks anything resembling this, and this oversight has yet to be addressed. It is this deficiency which our paper seeks to remedy.

We propose that the Concepts feature is incomplete without constrained overload set for usage, thus satisfying the third requirement of any interface-like abstraction. It is vital that we explore this issue.

We recognize that some complexity in the space of constrained generics will always be present, but we feel that it is best to offload this complexity to the author of a concept rather than to the implementor of a constrained function. This is because we believe that fewer concept authors will exist than concept "users". Additionally, the level of expertise of a concept author is inherently higher than the intended audience of constrained functions. In the worst case scenario, a naive definition of a concept will merely result in a few missed opportunities for more suitable overloads to handle move semantics, avoid conversions, and other shenanigans

Further, the introduction of concept replacer machinery facilitates a type-like feel to concepts in constrained contexts without actually putting pressure onto the type system itself. For some cases, the implicit generation of constraint replacers may be inappropriate; however, we feel that those cases will be somewhat infrequent. This concept replacer machinery is vital as it preserves the semantics of code across various refactoring patterns. The automatic generation of these concept replacers helps to ease the burden on making existing code useful to constrained functions.

Objections, Questions, and Concerns

Q: Isn't this just C++0x Concepts with definition checking all over again?

A: No. C++0x Concepts used mechanisms and techniques which are drastically different to the solution we have proposed. In the original C++0x Concepts, the compiler was required to create invisible wrapping types (Concept Maps) and create inline shimming functions, and to calculate the set of archetype types for a Concept. C++0x Concepts were difficult to inline and relied upon the optimizer to eliminate the overhead imposed by these automatically generated constructs. C++0x concepts also provided "definition checking", which was desirable in that a template function could be checked for correctness before instantiation. C++0x concepts used the "Concept Map" and archetype computations to decide whether a template would be correct. This compiletime computation of an archetype runs into a manifestation of the halting problem, which makes generalizing this solution to user defined concepts a problematic proposition at best.

By contrast, our solution does not involve the generation of such machinery. We require no generation of any adaptors, maps, or proxies. Instead, we propose altering and refining the lookup rules to further obey the restrictions imposed by Concepts in a manner similar to what is already in the existing design. We feel that this is appropriate, because Concepts already requires some alteration to the lookup rules, and our design appears to be consistent with the general lookup rule restrictions thereby imposed. Concept restrictions, in their current form, are enforced in C++ through lookup rules, not through any other mechanism. Our solution merely adds more rules to the set of lookup rules employed in concept processing. This also means that our solution is not capable of providing definition checking -- the lookup rules that this solution alters are those which occur during the second phase of name lookup, after template instantiation.

Q: Will I be able to call internal helper functions to my constrained function using an unqualified name?

A: Yes. We place no restrictions on the calling of functions in namespaces that are unrelated to the concept used in a constraint. The namespaces associated with the types that are constrained are also still searched, but only the names which are necessary (in some fashion) to meet the requirements of the concept are considered to be viable. In some sense this is the existing Concepts restriction on calling a constrained function applied in reverse -- constraints restrict which functions are called based upon their arguments. The current restriction prevents calling a function which is not prepared to accept a type. Our refinement prevents calling a function which is not presented as part of the requirements on a type. This example is illustrated in our "at scale" example, and it is one of our primary motivations.

Q: Isn't your real problem with {ADL, const vs. non-const overloads, overload resolution, dependent lookup, etc.} and not with the lookup rules of Concepts today?

A: Absolutely not. We have examples of unexpected selection of operation each and every one of these cases. We are not convinced that our problem is with every single one of the above aspects of the language. There are some cases which will be redundantly resolved by improving those aspects of the language; however, many problem cases within each of these domains still remain. This is especially true of ADL functions. ADL functions are intended to be part of the interface of a class; however, a constrained value is also a constrained interface.

Q: How do I actually invoke ADL functions that I want invoked in my constrained functions?

A: ADL functions on an object which are actually part of the interface defined by the concept that constrains the calling function are still preserved as part of the viable overload set. This makes them likely to be the best match for an unqualified call, unless a more specific overload is available in some other reachable namespace. If you wish to call ADL functions which are not part of the constraint, then one always has two viable options. The first is to use a direct "using std::foobar" approach, and the second is to adjust the definition of a Concept to include this ADL operation.

Q: What about calling efficient swap on an Assignable?

A: This is actually a special case of the above concern. In this case, there are at least two viable options. The first is to add `swap( a, b )` to the requirements of the Assignable concept. The second is to make `std::swap` have an overload which accepts a value which is a model of the Swapable concept. An unqualified call to `swap` after the traditional `using std::swap;` declaration will invoke that Swapable overload, thus giving the correct behavior. In addition, this has the added benefit of making any direct call to `std::swap` in any context always take the best overload! Although this library change is not proposed by this paper, the authors would strongly support such a change.

Q: Is this going to give me better error messages?

A: Although this is highly dependent upon the details of an implementation, it is possible that better error messages would be possible under this proposal. The instantiation of a template which requires some specific operation which is not part of a concept should give a better error message -- something along the lines of "Function not found during constraint checking."

Q: What about implicit conversions needed in calling a function which is part of a concept?

A: Because the function selected by overload resolution must be part of the operations necessary to satisfy the constraints of the specified concept, all implicit conversions which are necessary to invoke that function are also part of the operations necessary to satisfy that concept. This means that any set of implicit conversions provided by a type which are necessary to invoke a selected overload should be "whitelisted" for use in constrained functions when calling that overload.

Q: Will I be able to invoke arbitrary operations on my constrained parameters which are not part of the concept?

A: Any function which is available directly in the namespace (directly or via a `using` statement) of a constrained function may be called. Any function which is a template but is not constrained will be called as an unconstrained function. In that context of an unconstrained template, any function may be invoked as normal. We also propose that an intrinsic cast-like operation could be added which will revert a constrained variable to an unconstrained variable, to permit calling functions in an unconstrained fashion for various purposes.

Design Considerations

Any design that proposes to change lookup rules should not invalidate code written under those rules today. Because of this, we do not propose that the lookup rules should be changed when evaluating names within a "classical" template context. However, we see a number of opportunities to apply our modified lookup scheme:

Terse syntax constrained functions

The terse syntax intends to open generic programming to a wider audience, as discussed earlier. We feel that it is obvious that such terse syntax functions be subject to rules which provide a more intuitive result. Therefore we suggest that any terse syntax considered by the committee must have the intuitive semantics provided by our proposal.

Template syntax constrained function

All constrained function templates

Any expansion of a terse syntax from the terse form into a "canonical" production of a constrained template function declaration could automatically have these rules applied. This seems fairly obvious in many respects, because the purpose of Concepts is to afford better selection of applicable functions in name lookup and overload resolution. When a user writes a constrained function, even using template syntax, he or she is explicitly choosing to have the semantics of Concepts applied to their function. Therefore, it seems a reasonable choice to make every constrained function obey these lookup rules.

Opt-in for these rules as part of the definition of a constrained function template

A user trying to modernize a code base by adding constraints to existing template functions, may wind up causing subtle changes in the semantics and or ODR violations. Additionally, the template expert is already intimately familiar with the consequences of C++'s unintuitive lookup rules in templates and may wish to leverage the semantics afforded by these rules in the implementation of his template function -- he only wishes to constrain the callers, but not himself.

Therefore it may be necessary to control the application of this modified rule through the use of a signifying keyword. We propose `explicit template< ... >` as this syntax, as it reads reasonably well and clearly indicates intent. Although this proposed syntax uses the `template` keyword, which is already indicative of potential lookup dangers, it eschews the pitfalls for the `template< ... >` case.

Regardless of whether the new lookup rules are opt-in or opt-out, the language loses no expressivity. It is possible to choose the opposite alternative through other syntax.

Behavior of These Rules Under Short Circuit of Disjunction

We preserve the viability of overloads which are found on either branch of a disjunction, because a user would reasonably expect these overloads to be available if those constraints are satisfied. For branches of a disjunction which are not satisfied, those overloads will be unavailable, as the constraint wasn't satisfied. This seems to result in a viable overload set which most closely conforms to user expectations. The overall compile-time cost of this added checking should be proportional to the overall cost of treating each disjunction as a conjunction for this feature.

Directions for Future Investigation

It is somewhat disappointing a lambda passed into an algorithm is not able to get the constraint replacement mechanism in this proposal to inform it about the constraints on the parameter the lambda accepts. This is to be expected, from the nature of this proposal, as everything that happens for concept reintroduction happens within the constrained context. The lambda that is defined therein has exposure to those constraints, so the programmer can use that exposure to capture the knowledge of the constraints into the definition of the lambda. This is the only way, under a system such as this, that a lambda can gain benefits from concept reintroduction.

We could potentially explore another keyword (perhaps using this `typename< ... >` for this case or something similar) which would decorate a parameter, and the `< ... >` component would be used to inform the compiler of the signature of a function-like parameter to a function. This same kind of concept replacer, but for arguments would then be available for generating a wrapper lambda around any function-like object wherein that wrapper lambda imputes the constraint which was described in the `< ... >` of this hypothetical keyword. The idea would be that:

```
[] ( auto for concept &x ) { f( x ); }
```

would get wrapped in another lambda, before being passed to an algorithm. Like this:

```
[( __concept_described_in_our_hypothetical_keyword__ &&__x ) -> decltype( auto )  
{  
    return [] ( auto for concept &x ) { f( x ); }( __x );  
}]
```

This wrapper lambda would enforce the concept constraint itself, before forwarding control to the inner lambda, which would deduce the constraint from the wrapper lambda. Such an entity would correctly capture the constraint information at the definition site and package it up for use at the calling site. Since the type of the lambda is bound to the function in which it is defined, when the caller of an algorithm passes a lambda, he gets a distinct instantiation of that algorithm anyhow, thus not requiring mucking about in nasty ODR violation repercussions.

One downside of this approach is that it resembles concept maps very much, as we're building an actual wrapper object that has to be generated which could have impact on runtime performance. Some compilers may be able to eliminate the intermediate forwarding lambda, but not all of them.

Another downside of this approach is that it requires all algorithm-like functions to go to lengths to define the signature of their function-like parameters. Some implicit deduction may be possible here; however, there seems to be no clean way to do this. One way is to bless a special `std::functional< ... >` concept with the ability to have such deduction (but that is just explicit syntax by another form). Another way would be to analyze a concept, but it seems that a lot of it would be special cased upon

the function-like parameter in question being a concept which only has some lifecycle methods and an `operator() ( ... )` overload. Such a special case rule that certain expressions, and only certain expressions exist would make this work fine for lambdas being passed as function objects, but for little else. Consequently, we are content to delay this kind of investigation for a later date.

Our investigation of this kind of constraint has led us to observe that in a fully constrained generic function, a non-member function which is called from within the constrained function must be found in at least one of the constraints' definitions or be found during Phase 1 lookup of names in a template. Therefore, it may be possible to issue useful compiler diagnostics if a function is not found during Phase 1 and that function is not mentioned in any of the concept arguments. Such situations are constrained functions that almost certainly contain an error. There may be other kinds of diagnostic for obvious error cases which can be performed during the first phase of template compilation. We would like to investigate some of these possible early diagnostics in future proposals; however, we are content to leave such diagnostics as a Quality of Implementation issue at this time.

Conclusions

In P0726R0 it was asked if Concepts improve upon the expressivity of C++17. The response from EWG was mixed. The fact is that, although Concepts are more approachable and readable than C++17 `std::enable_if` predicates, they do not provide any new expressivity to the language, nor do they provide any facility for making templates actually easier to write.

Since that time, there have been numerous papers and proposals addressing the space of concepts syntax, especially regarding function declarations. We set out to address a different aspect of the concept space and address the implicit question posed by P0726R0 -- "Can concepts provide new expressivity?" We found that there were potentially surprising aspects to the semantics of constrained functions, so we investigated what the perceptions and preferences of non-experts were. So we set out to answer that and see if we could also answer "Can constrained functions have semantics which are more intuitive to the non-expert?".

In P0782R0 we made the case that there is a lot of semantic behavior of constrained functions which a non-template expert would find extremely confusing. In this first revision we explored the possibility of making generic functions more non-expert friendly by bringing their behavior more in line with the expectations of our surveyed non-experts. We found that this received strong enough positive support to keep investigating this line of research. We then set out to solve the problems that were manifest in the simple case of honoring a concept's constraints when calling a function. We strove to avoid the traps, pitfalls, dead-ends, and controversies which have beset other proposals in this space by taking a novel approach.

In P0782R1, our second revision of this paper, we explored a set of modifications to the overload resolution rules which effected the desired semantics that non-experts expected. This set of overload resolution rules was sufficient to handle simple cases of function invocation but clearly required some modifications and extensions to permit it to have non-surprising behavior in all circumstances and under refactorings. EWG support for this avenue of research was extremely strong, almost unanimous. We were given guidance to make these kinds of overload resolution rules behave as expected in various circumstances, such as STL algorithm calls, range-based for loops, and through type-identity circumstances. Despite these shortcomings, we (and others) found our approach promising as it successfully navigated away from the morass of problems that plagued other efforts at enforcing constraints within constrained functions.

We now, in this revision, have greatly expanded our proposal, introducing a whole new battery of language features specifically targetted at preserving compiletime constraint knowledge in an instantiation-safe way. This new constraint preserving machinery also comes with a set of rules for the implicit generation of such machinery in a vast array of cases. The implicit generation of this machinery is reminiscent of the implicit generation of deduction guides. The implicit generation of deduction guides made CTAD a much more usable and easy to deploy feature. It made the process of making the standard library ready for CTAD much easier by having mostly sensible defaults from the get-go. We have learned from this example and have provided a set of implicit generation of concept reintroduction guides (constraint replacers) which, although there is still some room for debate and discussion over what the best defaults are, will provide a much more simple path forward to adoption for libraries and existing code.

In our on-paper experimentation with this expanded proposal's rules we've found its semantics to be almost as intuitive as-if we'd had concepts in our type system all along, yet without the obvious burdens incurred by baking constraint information into the type system. The few times that the proposal's rules surprise us are when we forget that we cannot preserve constraint information for use in a lambda defined in constrained contexts which is invoked in some (possibly) unconstrained context. The use of `auto` for concept everywhere that `auto` is used in modern codebases causes hard-errors in the most problematic case, and we are confident that compiler warnings for vanilla `auto` can be used to cause equally hard errors for the `[]( auto &&x ){ ... }` case as well. Further, writing the known constraint on the lambda at the definition site becomes simpler with a mechanism like `decltype` for concept. Even with our intentionally ugly keyword choices, the system seems intuitive. The few non-experts in C++ we've shown these rules to (with cleaner keyword choices) find the rules intuitive enough to grasp and

find the results of their application about as easy to reason about as types and interfaces. To that end, we believe that this proposal satisfies our original goals, "Can concepts provide new expressivity?" and "Can constrained functions have semantics which are more intuitive to the non-expert?".

Although we were unable to fully attain the "viral through auto and function call" request, we believe that we've presented something which is a reasonable compromise -- the syntax for usage (once a set of better names for things like `auto` for concept are decided) should be relatively intuitive, and the need to change code in the problematic cases is rather infrequent and trivial when needed. We have outlined a possible future direction to attain the "viral through function call" request from Rapperswil, but the facilities provided in this paper already meet the criteria of easy to use, due to the implicit generation rules.

In the case of the "viral through function call" for lambda instantiation, our initial exploration of future work to make this possible leads us to believe that this could be added at a future date, after this change, in a non-backwards-breaking way.

We feel, now more than ever, that this revised proposal's adoption will give a new dimension to the Concepts feature which will enable the simple expression of C++ generic code in a much safer and more readable style than C++17 or the current Concepts design. We believe that the resulting language is much more intuitive for beginners and non-experts alike. We believe that the behavior of constrained functions under this proposal is equally intuitive and useful to C++ experts with deep template knowledge.

Revision History

P0782R0 - A Case for Simplifying/Improving Natural Syntax Concepts (Erich Keane, A.D.A. Martin, Allan Deutsch)

The original draft explained the general problem with a simple example. The motivation was to help attain a better consensus for "Natural Syntax" Concepts. It was presented in the Monday Evening session of EWG at Jacksonville, on 2018-03-12. The guidance from the group was strongly positive:

SF: 10 - F: 21 - N: 22 - A: 7 - SA: 1

P0782R1 - Constraining Concepts Overload Sets (A.D.A. Martin, Erich Keane)

The revised paper explains a solution to the specific problem of getting expected overload resolution out of an individual function call in a constrained context. It was presented to EWG during a day session at Rapperswil, in 2018. The guidance from the group was strongly positive to continue to pursue work in this direction:

SF: 20 - F: 11 - N: 8 - A: 1 - SA: 1

The group also requested and encouraged investigation into a few things:

Make concepts "viral" through function calls into templates and in `auto` such that some problematic examples are corrected:

SF: 10 - F: 10 - N: 8 - A: 1 - SA: 2

Make the overload resolution rule also apply to a fully qualified name, not just members and unqualified names:

SF: 11 - F: 12 - N: 6 - A: 0 - SA: 0

Acknowledgements

The authors would like to thank Nathan Myers, Allan Deutsch, Hal Finkel, Laura Reznick, Lisa Lippincott, Gabriel Dos Reis, Justin McHugh, Herb Sutter, Faisal Vali, Mike Gorbavitsky, and numerous others for their research, support, input, review, and guidance throughout the lifetime of this proposal. Without their assistance this would not have been possible.

References

P0726R0 - "Does Concepts Improve on C++17?"

P1079R0 - "A minimal solution to the concepts syntax problems"

P0745R1 - "Concepts in-place syntax"

P1013R0 - "Explicit concept expressions"

P1086R0 - "Natural Syntax: Keep It Simple"

P1084R0 - "Today's return-type-requirements Are Insufficient"